

I2S Microphone Recording and U-Disk Playback

I2S Microphone Recording and U-Disk Playback

- 1. Audio Recording Overview
- 2. Core Concepts
 - 2.1 I2S Receive Mode
 - 2.2 ES8311 ADC Mode
 - 2.3 PSRAM Large-Capacity Buffering
 - 2.4 Wear Leveling + FAT File System
 - 2.5 USB MSC Storage Ownership Switching
- 3. Project Architecture and Flow
- 4. Hardware Dependencies
- 5. Source Code Details
 - 5.1 main.c — Program Entry Point
 - 5.2 record_ctrl.c — Recording Main Controller
 - 5.3 app_speech.c — Audio Hardware Initialization
 - 5.4 app_speech.c — USB Storage Initialization
 - 5.5 wav_handler.c — WAV File Read/Write
 - 5.6 led_ctrl.c — WS2812 LED Status Feedback
- 6. Operation Flow
 - 6.1 menuconfig Configuration
 - 6.2 Building and Flashing
 - 6.3 Operation Steps
 - 6.4 Edge Cases
- 7. Common Problems

1. Audio Recording Overview

Audio recording is a common requirement in embedded products, widely used in smart voice assistants, voice recorders, security surveillance, conference recording, intercom systems, and other scenarios. Mastering I2S audio acquisition, PSRAM large-capacity buffering, FAT file system storage, and USB MSC simulated USB drive export are the core technical paths for implementing recording products.

This project implements a BOOT button-based audio recorder, which collects PCM audio data from the ES8311 audio codec (ADC mode) through the I2S RX channel, buffers it to PSRAM and saves it as a WAV file, and simulates a USB drive through USB MSC (Mass Storage) protocol, supporting direct PC reading of recording files.

Core capabilities:

Function	Description
Audio Acquisition	ES8311 ADC, 16-bit / 16kHz / Mono, Microphone gain 40dB
Data Buffering	PSRAM (8MB), max 60 seconds recording (1920KB)

Function	Description
File Storage	SPI Flash wear leveling + FAT file system
File Export	TinyUSB MSC simulated USB drive, PC plug-and-play
Button Control	BOOT button (GPIO0): Short press to start/stop recording
LED Feedback	WS2812 RGB LED: Green=Recording, Rainbow=Save complete

2. Core Concepts

2.1 I2S Receive Mode

Contrary to the playback project, recording uses the **I2S RX channel**. ESP32 acts as the I2S master to generate BCLK and WS, and the ES8311 ADC sends PCM data to ESP32 through the DIN pin via these clock signals.

Signal Direction	Playback Mode (TX)	Recording Mode (RX)
MCLK	ESP32 → ES8311	ESP32 → ES8311
BCLK	ESP32 → ES8311	ESP32 → ES8311
WS	ESP32 → ES8311	ESP32 → ES8311
DOUT	ESP32 → ES8311	Not used
DIN	Not used	ES8311 → ESP32

Key difference: In recording mode, `I2S_DIN = GPIO17`, `I2S_DOUT = GPIO_NUM_NC` (not used).

2.2 ES8311 ADC Mode

ES8311 is configured in **ADC mode** (`ESP_CODEC_DEV_WORK_MODE_ADC`):

Parameter	Value
Work Mode	ADC only
Sample Rate	16kHz
Bit Depth	16-bit
Channel	Mono
MIC Gain	40 dB
DAC Output	Muted (to prevent idle noise)

```
es8311_codec_cfg_t es8311_cfg = {
    .codec_mode = ESP_CODEC_DEV_WORK_MODE_ADC, // ADC mode
    .use_mclk   = true,                       // Use MCLK
};
// Microphone gain 40dB
esp_codec_dev_set_in_gain(record_dev_, 40);
```

2.3 PSRAM Large-Capacity Buffering

ESP32-S3 has built-in **8MB PSRAM**. This project uses `heap_caps_malloc` to allocate recording buffer from PSRAM:

```
// Allocate 60s x 16kHz x 2B = 1920KB from PSRAM
int16_t *audio_buf = heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_SPIRAM);
```

Parameter	Value
Sample Rate	16000 Hz
Bit Depth	16-bit
Channel	1 (mono)
Max Duration	60 seconds
Buffer Size	1920 KB
Memory Source	PSRAM (does not occupy internal SRAM)

Using PSRAM instead of internal RAM prevents the recording buffer from occupying the limited 512KB internal SRAM.

2.4 Wear Leveling + FAT File System

SPI Flash has limited erase/write cycles (~100,000 times). Frequent direct writes will shorten its lifespan.

Technology	Function
Wear Leveling (WL)	Maps logical sectors to different physical sectors, balancing erase/write distribution
FAT File System	Standard FAT12/16 format, directly recognized by PC
TinyUSB MSC	Implements USB Mass Storage Class, simulates USB drive

Data flow:

```
Recording PCM data → heap_caps_malloc(PSRAM) → Buffer complete
→ wav_save_file() → FAT FS → wear Leveling → SPI Flash
→ storage_mount_usb() → TinyUSB MSC → PC recognizes as USB drive
```

2.5 USB MSC Storage Ownership Switching

The storage device switches between **APP side** (firmware read/write) and **USB side** (PC access):

State	Owner	Purpose
Recording	APP side	Firmware writes WAV file to <code>/data/</code>
Save complete	USB side	PC recognizes as USB drive, reads recording files
Before starting new recording	APP side	Firmware deletes old files

```
storage_mount_app(); // Switch storage to APP side
storage_mount_usb(); // Switch storage to USB side
```

3. Project Architecture and Flow

```
app_main()
└ record_ctrl_init()
  └ led_init()           ← WS2812 LED initialization
  └ Codec_I2S_init()     ← Recording chain initialization
    └ i2c_bus_init()      I2C master → ES8311 control interface
    └ i2s_init()          I2S RX channel (master/Philips/Mono)
    └ es8311_codec_init() ES8311 ADC mode, MIC gain 40dB
    └ power_on_codec()    NS4150 PA enable (provides MIC bias)
  └ storage_usb_init()   ← FAT + USB MSC initialization
    └ esp_partition_find_first() Find FAT partition
    └ wl_mount()          wear leveling
    └ tinyusb_msc_new_storage_spiflash() Create MSC storage instance
    └ tinyusb_msc_set_storage_mount_point(APP) Mount to APP side
    └ tinyusb_driver_install() Install TinyUSB driver
  └ xTaskCreate(button_task) ← Button polling task (~100Hz)

button_task (FreeRTOS Task):
while(1):
  Detect BOOT button falling edge (debounce 20ms)
  └ STATE_IDLE → Create record_task, enter STATE_RECORD
  └ STATE_RECORD → Set STATE_IDLE, stop recording

record_task (FreeRTOS Task):
└ heap_caps_malloc(PSRAM, 1920KB)
└ while(g_state==RECORD):
  └ app_speech_record() → I2S RX read PCM
  └ storage_mount_app() → Mount to APP side
  └ wav_save_file()      → Write to /data/record.wav
  └ storage_mount_usb()  → Mount to USB side, PC can read
```

4. Hardware Dependencies

Hardware connections:

Pin	Function	Connection Target
GPIO42	I2C SCL	ES8311 SCL
GPIO41	I2C SDA	ES8311 SDA
GPIO15	I2S MCLK	ES8311 MCLK
GPIO16	I2S BCLK	ES8311 BCLK
GPIO18	I2S WS	ES8311 LRCK
GPIO17	I2S DIN	ES8311 SDOUT (ADC)
GPIO7	PA_EN	NS4150 PA enable (provides MIC bias power)
GPIO48	WS2812 Data	RGB LED
GPIO0	BOOT button	Active low
USB D+/D-	USB OTG	Connect to PC (USB drive mode)

Required components: `esp_codec_dev` / `es8311` / `driver/i2s_std` / `driver/i2c_master` / `led_strip` / `wear_leveling` / `FATFS` / `TinyUSB` / `FreeRTOS`

5. Source Code Details

Complete source code: `Source Code` → `ESP-IDF` → `6.Audio` → `Audio_Recording`

The system consists of 4 source files, read in call chain order: `main.c` startup → `record_ctrl.c` controls recording flow → `app_speech.c` provides audio hardware and USB storage capabilities → `wav_handler.c` and `led_ctrl.c` handle file packaging and status display.

This project uses the following third-party libraries, all from the ESP-IDF Component Registry, not built-in IDF. Dependencies must be declared in `main/idf_component.yml` before building, otherwise compilation will fail to find the corresponding header files:

Component	Version	Purpose	Reference Tutorial
<code>espressif/esp_codec_dev</code>	<code>^1.5.10</code>	Audio codec abstraction layer, encapsulates ES8311 and I2S operations	—

Component	Version	Purpose	Reference Tutorial
<code>espressif/esp_tinyusb</code>	<code>^2.0.1~1</code>	TinyUSB component, implements USB MSC simulated USB drive	3.Serial Communication and Storage System->6.USB Mass Storage Simulation Experiment->USB Mass Storage Simulation Experiment
<code>espressif/led_strip</code>	<code>^3.0.3</code>	WS2812 RGB LED driver	2.Basic Peripherals->10.RGB LED Effects

```
## IDF Component Manager Manifest File
dependencies:
idf:
  version: '>=4.1.0'
espressif/esp_codec_dev: ^1.5.10 # Audio codec
espressif/esp_tinyusb: ^2.0.1~1 # USB MSC drive
espressif/led_strip: ^3.0.3 # WS2812 RGB LED
```

After declaration, execute `idf.py build`, and the component manager will automatically download each component to the `managed_components/` directory.

5.1 main.c — Program Entry Point

`app_main()` only calls `record_ctrl_init()`, which starts the audio hardware, storage system, and button polling task in one go.

```
#include "record_ctrl.h"

void app_main(void)
{
    record_ctrl_init(); // start all subsystems
}
```

5.2 record_ctrl.c — Recording Main Controller

`record_ctrl_init()` initializes LED → audio chain → USB storage → button polling task in sequence. The BOOT button drives a two-state state machine: switching between idle (IDLE) and recording (RECORD).

State machine:

```
typedef enum {
    STATE_IDLE, // Idle
    STATE_RECORD // Recording
} work_state_t;
```

Button polling task (~100Hz, debounce 20ms):

```
static void button_task(void *arg)
```

```

{
    gpio_config_t gpio_conf = {
        .pin_bit_mask = BIT64(BUTTON_PIN),
        .mode          = GPIO_MODE_INPUT,
        .pull_up_en    = 1,
    };
    gpio_config(&gpio_conf);

    while (1) {
        if (gpio_get_level(BUTTON_PIN) == 0) { // Pressed
            vTaskDelay(pdMS_TO_TICKS(20)); // Debounce
            if (gpio_get_level(BUTTON_PIN) == 0) { // Confirmed
                if (g_state == STATE_IDLE) {
                    // Start recording task
                    xTaskCreate(record_task, "record", 8192, NULL, 5, NULL);
                } else if (g_state == STATE_RECORD) {
                    g_state = STATE_IDLE; // Stop
                }
                while (gpio_get_level(BUTTON_PIN) == 0) {
                    vTaskDelay(pdMS_TO_TICKS(10)); // wait release
                }
            }
        }
        vTaskDelay(pdMS_TO_TICKS(10)); // Poll interval
    }
}

```

Recording task — PSRAM buffer → Loop read PCM → Save WAV:

```

static void record_task(void *pv)
{
    // Allocate 1920KB from PSRAM
    int16_t *audio_buf = heap_caps_malloc(REC_BUF_LEN, MALLOC_CAP_SPIRAM);

    g_state = STATE_RECORD;
    g_rec_len = 0;
    led_set(0, 150, 0); // Green LED

    while (g_state == STATE_RECORD && g_rec_len < REC_BUF_LEN) {
        int ret = app_speech_record(&audio_buf[g_rec_len / 2], FRAME_SIZE);
        if (ret == 0) {
            g_rec_len += FRAME_SIZE;
        } else {
            fail_count++;
            // Stop after 50 consecutive fails
            if (fail_count > 50) break;
        }
    }

    led_off();
    if (g_rec_len > 0) {
        storage_mount_app(); // Switch to APP side
        unlink(WAV_FILENAME); // Remove old file
    }
}

```

```

        wav_save_file(WAV_FILENAME, (uint8_t *)audio_buf, g_rec_len);
        storage_mount_usb();           // Switch to USB side
        led_rainbow();                 // Rainbow celebration
    }
    free(audio_buf);
    vTaskDelete(NULL);
}

void record_ctrl_init(void)
{
    led_init();                       // LED
    Codec_I2S_init();                 // Audio pipeline
    storage_usb_init();               // USB storage
    xTaskCreate(button_task, "button", 4096, NULL, 5, NULL);
}

```

FRAME_SIZE = 1600 bytes (800 samples × 2 bytes), LOG_INTERVAL = 20 (prints progress approximately once per second).

5.3 app_speech.c — Audio Hardware Initialization

`Codec_I2S_init()` provides the entry point for initializing the recording chain, internally calling I2C bus, I2S RX channel, and ES8311 codec initialization functions in sequence. This file also contains USB storage related functions (see section 5.4).

Related pins and parameter macros (defined in `app_speech.h`):

Macro	Value	Description
<code>I2C_SCL_PIN</code> / <code>I2C_SDA_PIN</code>	GPIO42 / GPIO41	I2C pins
<code>I2S_MCLK</code> / <code>I2S_BCLK</code> / <code>I2S_WS</code>	GPIO15 / GPIO16 / GPIO18	I2S clock pins
<code>I2S_DIN</code> / <code>I2S_DOUT</code>	GPIO17 / GPIO_NUM_NC	I2S data (RX only)
<code>EXAMPLE_SAMPLE_RATE</code>	16000	Sample rate (Hz)
<code>EXAMPLE_MCLK_MULTIPLE</code>	256	$MCLK = 16000 \times 256 = 4.096\text{MHz}$
<code>EXAMPLE_MIC_GAIN</code>	40	Microphone gain (dB)

Codec_I2S_init() — Initialization entry:

```

void Codec_I2S_init(void)
{
    i2c_bus_init();           // I2C master bus
    i2s_init();               // I2S RX channel
    es8311_codec_init();      // ES8311 ADC mode
    power_on_codec();         // PA enable (MIC bias)
}

```

i2s_init() — Create I2S RX channel (master / Philips / Mono):


```

static void i2s_init(void)
{
    i2s_chan_config_t chan_cfg = {
        .id          = I2S_NUM_0,
        .role         = I2S_ROLE_MASTER,
        .dma_desc_num = 8,
        .dma_frame_num = 480,
    };
    i2s_new_channel(&chan_cfg, NULL, &s_rx_chan); // RX only

    i2s_std_config_t std_cfg = {
        .clk_cfg = {
            .sample_rate_hz = EXAMPLE_SAMPLE_RATE,
            .mclk_multiple  = EXAMPLE_MCLK_MULTIPLE,
        },
        .slot_cfg = I2S_STD_PHILIPS_SLOT_DEFAULT_CONFIG(
            I2S_DATA_BIT_WIDTH_16BIT,
            I2S_SLOT_MODE_MONO),
        .gpio_cfg = {
            .mclk = I2S_MCLK, .bclk = I2S_BCLK, .ws = I2S_WS,
            .din  = I2S_DIN,
            .dout = I2S_DOUT,
        },
    };
    i2s_channel_init_std_mode(s_rx_chan, &std_cfg);
    i2s_channel_enable(s_rx_chan); // Enable early
}

```

es8311_codec_init() — Configure ES8311 ADC mode (6-step initialization):

```

static void es8311_codec_init(void)
{
    // ① I2C control interface
    audio_codec_i2c_cfg_t i2c_cfg = {
        .bus_handle = s_i2c_bus,
        .addr       = (ES8311_ADDR << 1), // see footnote below
    };
    s_ctrl_if = audio_codec_new_i2c_ctrl(&i2c_cfg);

    // ② GPIO interface (PA manual control)
    s_gpio_if = audio_codec_new_gpio();

    // ③ I2S data interface (RX only)
    audio_codec_i2s_cfg_t i2s_cfg = { .rx_handle = s_rx_chan };
    s_data_if = audio_codec_new_i2s_data(&i2s_cfg);

    // ④ ES8311 codec instance
    es8311_codec_cfg_t es8311_cfg = {
        .codec_mode = ESP_CODEC_DEV_WORK_MODE_ADC,
        .ctrl_if    = s_ctrl_if,
        .gpio_if    = s_gpio_if,
        .pa_pin     = -1, // PA manual control
    };
}

```

```

        .use_mclk      = true,
        .master_mode = false,    // ESP32 is I2S master
        .mclk_div     = EXAMPLE_MCLK_MULTIPLE,
    };
    s_codec_if = es8311_codec_new(&es8311_cfg);

    // ⑤ Create and open device (16kHz / 16-bit / mono)
    esp_codec_dev_cfg_t dev_cfg = {
        .codec_if = s_codec_if,
        .data_if  = s_data_if,
        .dev_type = ESP_CODEC_DEV_TYPE_IN,
    };
    record_dev_ = esp_codec_dev_new(&dev_cfg);

    esp_codec_dev_sample_info_t fs = {
        .bits_per_sample = 16,
        .channel          = 1,
        .channel_mask     = ESP_CODEC_DEV_MAKE_CHANNEL_MASK(0),
        .sample_rate      = EXAMPLE_SAMPLE_RATE,
        .mclk_multiple    = EXAMPLE_MCLK_MULTIPLE,
    };
    esp_codec_dev_open(record_dev_, &fs);

    // ⑥ Set gain + mute DAC
    esp_codec_dev_set_in_gain(record_dev_, EXAMPLE_MIC_GAIN);
    s_codec_if->mute(s_codec_if, true);
}

```

About (`ES8311_ADDR << 1`): ES8311's 7-bit I2C address is `0x18`. The `audio_codec_new_i2c_ctrl()` function internally **right-shifts** the passed address back to a 7-bit address, so the caller must pass the left-shifted 8-bit form: `0x18 << 1 = 0x30`. If you directly write `.addr = 0x18`, the library internally shifts it right to `0x0C`, and the I2C bus will not find the device.

app_speech_record() — Read PCM (called by record_task):

```

int app_speech_record(void *buf, int len)
{
    return esp_codec_dev_read(record_dev_, buf, len);
}

```

5.4 app_speech.c — USB Storage Initialization

`storage_usb_init()` exposes the FAT partition on SPI Flash as a USB drive through the TinyUSB MSC protocol, allowing PC to read recording files. Storage ownership can switch between APP side (firmware write) and USB side (PC access).

storage_usb_init() — 5 steps to complete storage initialization:

```

void storage_usb_init(void)
{
    // ① Find FAT data partition
    const esp_partition_t *data_part = esp_partition_find_first(

```

```

        ESP_PARTITION_TYPE_DATA, ESP_PARTITION_SUBTYPE_DATA_FAT, NULL);

// ④ Mount wear leveling
wl_mount(data_part, &s_wl_handle);

// ⑤ Create MSC storage instance
tinyusb_msc_storage_config_t storage_cfg = {
    .mount_point      = TINYUSB_MSC_STORAGE_MOUNT_USB,
    .medium.wl_handle = s_wl_handle,
};
tinyusb_msc_new_storage_spiflash(&storage_cfg, &s_storage_hdl);
tinyusb_msc_set_storage_callback(storage_mount_changed_cb, NULL);

// ⑥ Mount to APP side, firmware can write
tinyusb_msc_set_storage_mount_point(
    s_storage_hdl, TINYUSB_MSC_STORAGE_MOUNT_APP);

// ⑦ Install TinyUSB driver
tinyusb_config_t tusb_cfg = TINYUSB_DEFAULT_CONFIG();
tusb_cfg.descriptor.device      = &descriptor_config;
tusb_cfg.descriptor.full_speed_config = msc_fs_configuration_desc;
tusb_cfg.descriptor.string      = string_desc_arr;
tusb_cfg.descriptor.string_count = 5;
tinyusb_driver_install(&tusb_cfg);
}

```

Storage ownership switching:

```

// Switch to USB side, PC sees a drive
void storage_mount_usb(void)
{
    tinyusb_msc_set_storage_mount_point(
        s_storage_hdl, TINYUSB_MSC_STORAGE_MOUNT_USB);
}

// Switch back to APP side, firmware can read/write
void storage_mount_app(void)
{
    tinyusb_msc_set_storage_mount_point(
        s_storage_hdl, TINYUSB_MSC_STORAGE_MOUNT_APP);
}

```

5.5 wav_handler.c — WAV File Read/Write

Raw PCM data cannot be directly recognized by PC players and needs to be encapsulated with a standard WAV header. `wav_save_file()` automatically generates a 44-byte header and then writes PCM data to disk.

```

esp_err_t wav_save_file(const char* filename,
                        uint8_t* audio_buffer, size_t audio_size)
{
    FILE* fp = fopen(filename, "wb");
    if (!fp) return ESP_FAIL;
}

```

```

wav_header_t header;
wav_create_header(&header, 16000, 16, 1); // 16kHz / 16-bit / mono
header.data_bytes = audio_size;
header.wav_size   = audio_size + sizeof(wav_header_t) - 8;

// 44-byte header
fwrite(&header, sizeof(wav_header_t), 1, fp);
fwrite(audio_buffer, 1, audio_size, fp);      // PCM data
fclose(fp);
return ESP_OK;
}

```

5.6 led_ctrl.c — WS2812 LED Status Feedback

Drives a single WS2812 LED using the `led_strip` component library (based on RMT peripheral), using colors to feedback system status: green light during recording, rainbow cycle on successful save.

```

void led_init(void)
{
    led_strip_config_t strip_cfg = {
        .strip_gpio_num    = LED_GPIO,
        .max_leds          = 1,
        .led_model         = LED_MODEL_WS2812,
        .color_component_format = LED_STRIP_COLOR_COMPONENT_FMT_GRB,
    };
    led_strip_rmt_config_t rmt_cfg = {
        .resolution_hz = 10 * 1000 * 1000,
    };
    led_strip_new_rmt_device(&strip_cfg, &rmt_cfg, &s_led);
    led_strip_clear(s_led);
}

void led_set(uint8_t r, uint8_t g, uint8_t b)
{
    if (!s_led) return;
    led_strip_set_pixel(s_led, 0, r, g, b); // pixel 0
    led_strip_refresh(s_led);
}

void led_rainbow(void)
{
    // 7-color cycle, 200ms each
    static const struct { uint8_t g, r, b; } colors[] = {
        {150, 0, 0}, // Green
        {60, 150, 0}, // Orange
        {150, 150, 0}, // Yellow
        {0, 150, 0}, // Red
        {150, 0, 150}, // Cyan
        {0, 0, 150}, // Blue
        {0, 120, 150}, // Purple
    };
};

```

```

    for (int i = 0; i < 7; i++) {
        led_strip_set_pixel(s_led, 0,
            colors[i].r, colors[i].g, colors[i].b);
        led_strip_refresh(s_led);
        vTaskDelay(pdMS_TO_TICKS(200));
    }
    led_strip_clear(s_led);
}

```

6. Operation Flow

6.1 menuconfig Configuration

This project depends on low-level functions such as PSRAM, custom partition table, FAT file system, and TinyUSB MSC. The following configuration options need to be enabled in menuconfig before compiling. For detailed operation procedures (including menuconfig screenshots) of common items such as partition table, TinyUSB MSC, Wear-Leveling, and FATFS long filename, please refer to **Section 4.2.1 in 3.Serial Communication and Storage System->6. USB Emulated Mass Storage Experiment**.

Configuration Path	Setting Item	Value	Description
Serial flasher config → Flash size	Flash size	16 MB	Match onboard 16MB SPI Flash
Partition Table → Partition Table	Custom partition table CSV	partitions.csv	Custom partition (with FAT data partition)
Component config → ESP PSRAM	Support for external PSRAM	Enable	Enable 8MB PSRAM, recording buffer source
Component config → ESP PSRAM	Allow malloc() in external memory	Enable	Allow memory allocation from PSRAM
Component config → FAT Filesystem	Long filename support	Heap	Support long filename
Component config → FAT Filesystem	Sector size	4096	Sector size matches Flash
Component config → FAT Filesystem	Prefer external RAM	Enable	FAT cache prefers PSRAM
Component config → Wear Leveling	Sector size	4096	Sector size matches Flash
Component config → TinyUSB	MSC Enabled	Enable	Enable USB mass storage
Component config → TinyUSB	MSC buffer size	8192	MSC transfer buffer size

The above configuration is written to the project's `sdkconfig.defaults`. After executing `idf.py set-target esp32s3`, IDF will automatically apply it. If compilation error reports that a certain feature is not enabled, please enter `idf.py menuconfig` and check each item one by one.

```
# Flash size
CONFIG_ESPTOOLPY_FLASHSIZE_16MB=y

# Custom partition table
CONFIG_PARTITION_TABLE_CUSTOM=y
CONFIG_PARTITION_TABLE_CUSTOM_FILENAME="partitions.csv"

# PSRAM / external RAM
CONFIG_SPIRAM=y
CONFIG_SPIRAM_USE_MALLOC=y

# FAT Filesystem
CONFIG_FATFS_LFN_HEAP=y
CONFIG_FATFS_SECTOR_4096=y
CONFIG_FATFS_ALLOC_PREFER_EXTRAM=y

# Wear Leveling
CONFIG_WL_SECTOR_SIZE_4096=y

# TinyUSB MSC
CONFIG_TINYUSB_MSC=y
CONFIG_TINYUSB_MSC_BUFSIZE=8192
```

6.2 Building and Flashing

ESP-IDF flashing flow: See `ESP-IDF Development Tutorial` → 1. Before Development → 3. Build and Flash.

6.3 Operation Steps

Step 1: Confirm hardware connections

This development board has ES8311 audio codec chip and MEMS microphone onboard. No external modules are needed. It can be used immediately after powering on.

Step 2: Serial output:

```
I (1500) app_speech: I2C bus init OK, SDA=41 SCL=42
I (1500) app_speech: I2S RX channel init OK, 16000 Hz / 16-bit / mono
I (1510) app_speech: ES8311 ADC recording init OK
I (1610) app_speech: PA amplifier enabled on GPIO7
I (1610) app_speech: =====
I (1610) app_speech:   Audio system ready (record only)
I (1610) app_speech:   I2C + I2S + ES8311 (esp_codec_dev)
I (1610) app_speech: =====
I (1620) app_speech: USB MSC storage init OK (APP side, path=/data)
I (1620) REC_CTRL: Recording control module init complete
I (1620) REC_CTRL: Ready: press BOOT to start/stop recording
```

```

I (1206) app_speech: I2C bus init OK, SDA=41 SCL=42
I (1206) app_speech: I2S RX channel init OK, 16000 Hz / 16-bit / mono
I (1206) I2S_IF: Auto get out handle 0 from in handle 0x3c070dfc when duplex mode
I (1226) ES8311: Work in Slave mode
I (1226) I2S_IF: No paired data, current mode: record
I (1226) I2S_IF: STD: RX, data_bit: 16, slot_bit: 16, ws_width: 16, slot_mode: MO
I (1236) I2S_IF: STD: RX, sample_rate_hz: 16000, mclk_multiple: 256, clk_src: 6
I (1256) Adev Codec: Open codec device OK
I (1266) app_speech: ES8311 ADC recording init OK
I (1366) app_speech: PA amplifier enabled on GPIO7
I (1366) app_speech: =====
I (1366) app_speech:   Audio system ready (record only)
I (1366) app_speech:   I2C + I2S + ES8311 (esp_codec_dev)
I (1366) app_speech: =====
I (1886) TinyUSB: TinyUSB Driver installed on port 0
I (1886) app_speech: USB MSC storage init OK (APP side, path=/data)
I (1896) REC_CTRL: Ready: press BOOT to start/stop recording
I (1896) REC_CTRL: Recording control module init complete
I (1906) main task: Returned from app main()

```

Step 3: Start recording

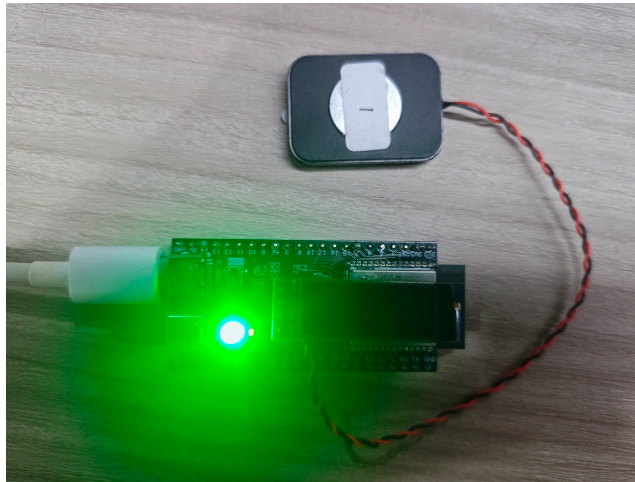
Short press BOOT button (GPIO0), LED turns green, recording starts. Serial port prints progress every second:

```

I (4200) REC_CTRL: ==== Recording started (max 60 s) ====
I (5200) REC_CTRL: Recording... 1 s
I (6200) REC_CTRL: Recording... 2 s
I (7200) REC_CTRL: Recording... 3 s
...

```

Speak towards the onboard MEMS microphone to capture audio.



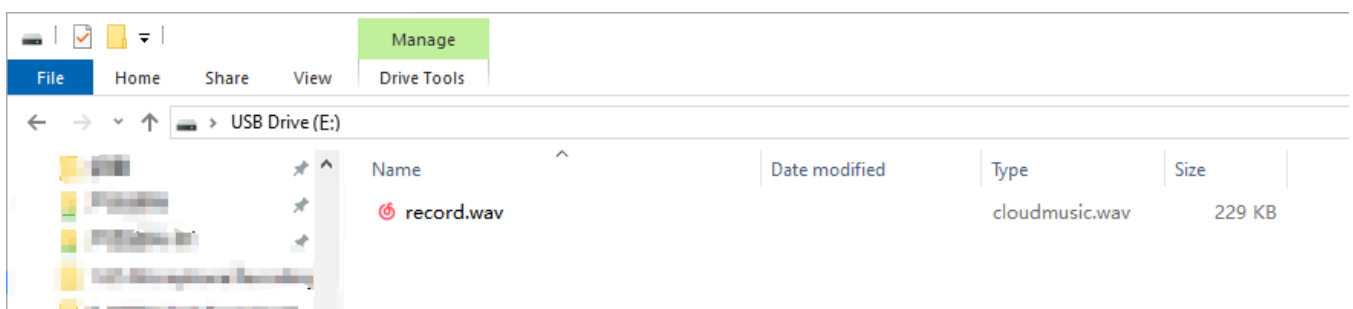
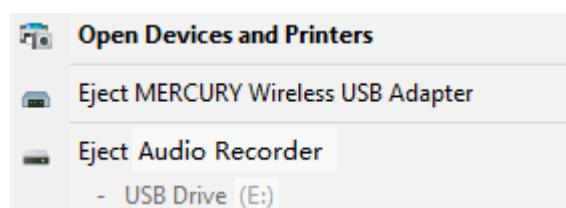
Step 4: Stop recording, save and export

Short press BOOT button again, recording stops. After LED turns off, file is written to Flash. After writing is complete, rainbow light turns on (7-color cycle), **indicating save is complete, you can change cable to retrieve files**. Serial output:

```
I (10800) REC_CTRL: Recording stopped by user
I (10800) REC_CTRL: ==== Recording stopped: 96000 bytes (3.0 s) ====
I (10810) REC_CTRL: Saving WAV file...
I (10820) wav: Save OK: /data/record.wav
I (10820) REC_CTRL: WAV saved: /data/record.wav (0.1 MB)
I (10820) app_speech: Storage switched to USB side, PC should see a USB drive
```

```
I (293366) REC_CTRL: ==== Recording started (max 60 s) ====
I (294156) REC_CTRL: Recording... 1 s
I (295156) REC_CTRL: Recording... 2 s
I (296146) REC_CTRL: Recording... 3 s
I (297166) REC_CTRL: Recording... 4 s
I (298156) REC_CTRL: Recording... 5 s
I (299146) REC_CTRL: Recording... 6 s
I (300166) REC_CTRL: Recording... 7 s
I (300416) REC_CTRL: Recording stopped by user
I (300466) REC_CTRL: ==== Recording stopped: 233600 bytes (7.3 s) ====
I (300466) REC_CTRL: Saving WAV file...
I (303836) wav: Save OK: /data/record.wav
I (303836) REC_CTRL: WAV saved: /data/record.wav (0.2 MB)
I (305236) app_speech: Storage mount point: USB
I (305236) app_speech: Storage switched to USB side, PC should see a USB drive
```

After seeing the rainbow light, unplug the USB cable from the **UART-USB port** (flashing serial port) and insert it into the **USB OTG port** (native USB, GPIO19/20). The PC will pop up a USB drive (volume label "Audio Recorder"). Open it to see `record.wav`:



Double-click to play and verify the recording effect. To record again, unplug USB, plug back into UART-USB for power, short press BOOT button (new recording will automatically overwrite old file).

6.4 Edge Cases

Scenario	Expected Behavior
Recording for 60 seconds (full)	Auto stop, save file
USB not plugged during recording	Normal recording and save, USB can still be read after connection
Repeated recording	New recording automatically deletes old file, writes new file

Scenario	Expected Behavior
No MIC input	Records silent file (all 0 or close to 0)

7. Common Problems

Symptom	Cause	Solution
PSRAM allocation failed	PSRAM not initialized or already occupied	Check <code>menuconfig</code> → Component config → ESP PSRAM
Recording file is 0 bytes	I2S RX did not receive data	Check I2S DIN pin (GPIO17) connection
MIC no sound/very low volume	MIC gain insufficient / ES8311 MICBIAS not enabled	Increase <code>EXAMPLE_MIC_GAIN</code> (max 42dB)
PC does not recognize USB drive	USB OTG mode not enabled / storage not switched	Confirm <code>storage_mount_usb()</code> has been called
WAV file corrupted	File write interrupted / insufficient flash space	Check FAT partition size, ensure enough space
PA has noise	DAC mute not effective	Check <code>s_codec_if->mute(true)</code>
SPI Flash lifespan short	Wear leveling not enabled	Confirm <code>wl_mount()</code> has been called
I2C initialization failed	Pin conflict or insufficient pull-up	Check I2C pin connections, enable internal pull-up